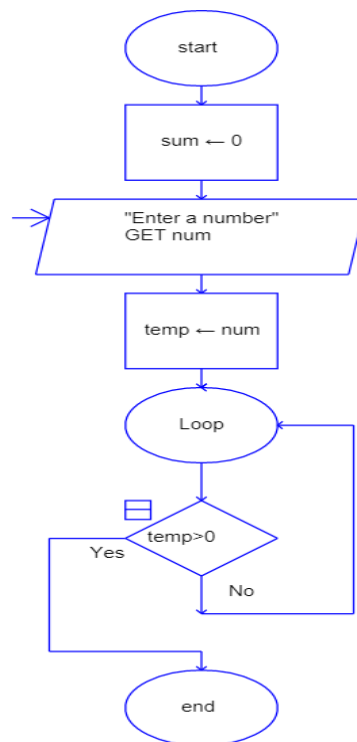


EXP: 01 - ARMSTRONG NUMBER

AIM

To design and implement a RAPTOR flowchart to check whether a given number is an Armstrong number or not.

FLOWCHART



SAMPLE INPUT

153

SAMPLE OUTPUT

153 is an Armstrong Number.

RESULT

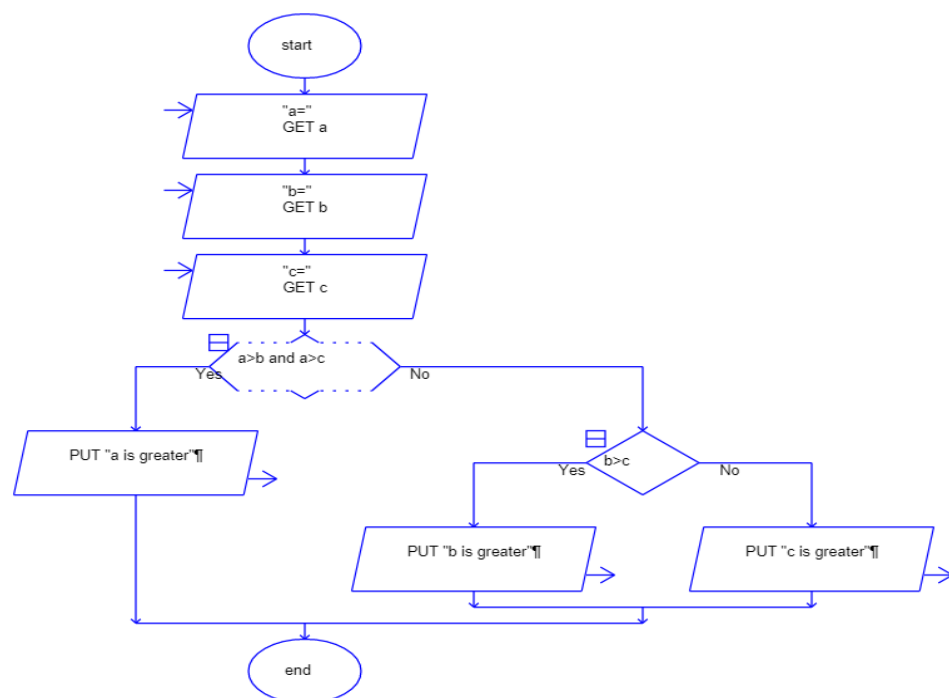
Thus, the RAPTOR flowchart was designed and executed successfully to determine whether the given number is an Armstrong number or not.

EXP:02-MAXIMUM OF THREE NUMBERS

AIM

To design and implement a RAPTOR flowchart to find the largest among three given numbers.

FLOWCHART



SAMPLE INPUT

A = 25 , B = 18 , C = 40

SAMPLE OUTPUT

Maximum Number = 40

RESULT

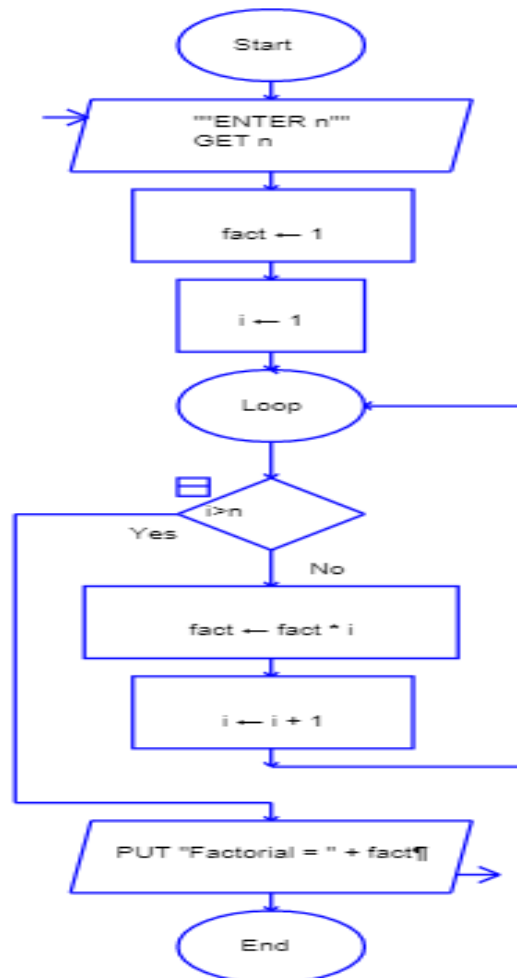
Thus, the RAPTOR flowchart was designed and executed successfully to find the maximum among three given numbers.

EXP: 03 – FACTORIAL

AIM

To design and implement a RAPTOR flowchart to find the factorial of a given number.

FLOWCHART



SAMPLE INPUT

5

SAMPLE OUTPUT

Factorial = 120

RESULT

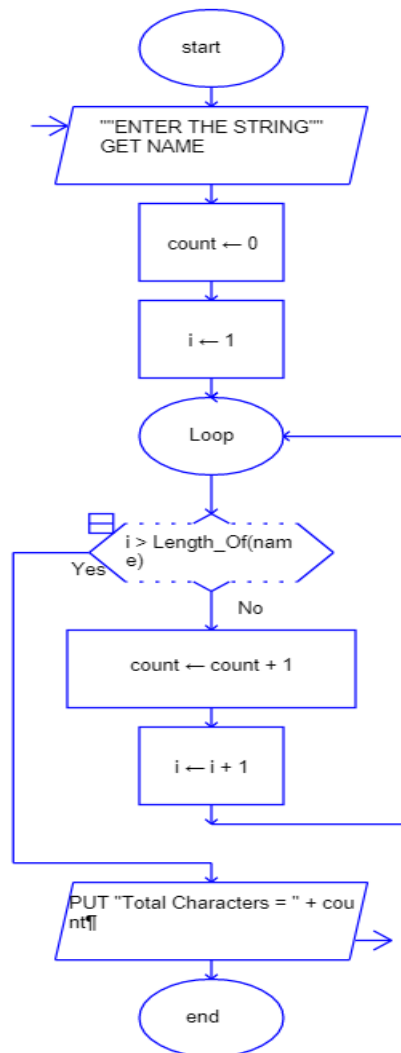
Thus, the RAPTOR flowchart was executed successfully and the factorial of the given number was displayed correctly.

EXP : 04 - String Handling

AIM

To create a RAPTOR flowchart that accepts a string from the user, counts the total number of characters present in the string, and displays the count.

FLOWCHART



SAMPLE INPUT

NAME = HELLO

SAMPLE OUTPUT

Total Characters = 5

RESULT

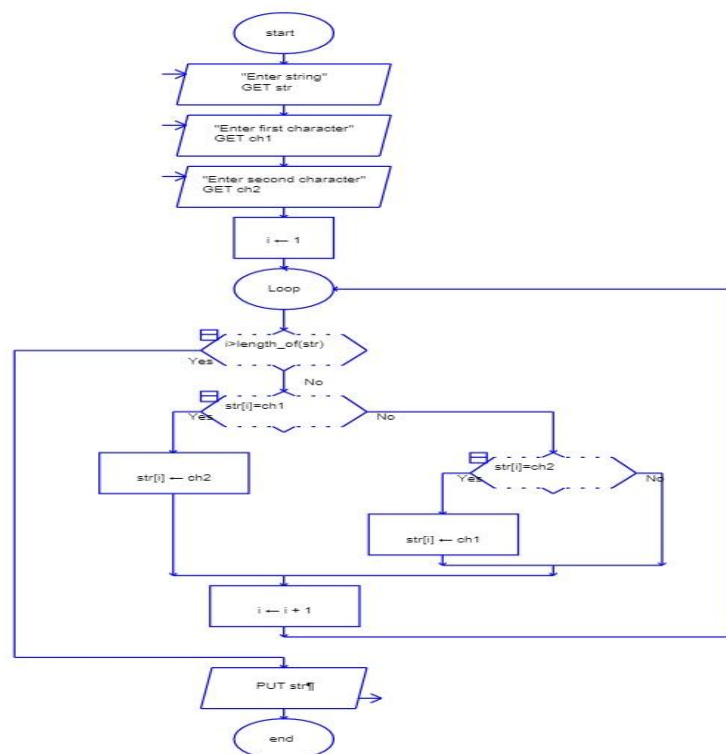
The flowchart was executed successfully. The program accepted a string from the user, counted the total number of characters in the string, and displayed the correct character count.

EXP 05 Lexicographically Smallest String Using Character Swap

AIM

To draw a RAPTOR flowchart that reads a lowercase string and finds the lexicographically smallest string by swapping all occurrences of any two characters at most once

FLOWCHART



SAMPLE INPUT :

ccad

SAMPLE OUTPUT

aacd

RESULT

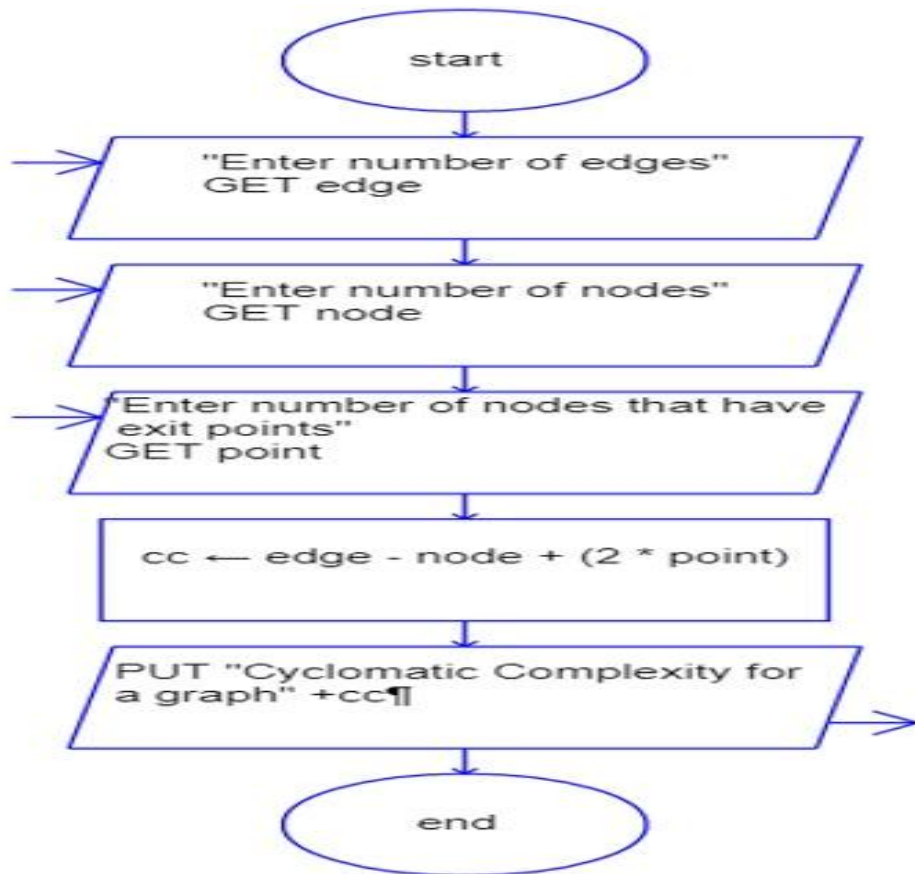
Thus, the RAPTOR flowchart for obtaining the lexicographically smallest string by performing at most one swap of two characters was successfully designed, executed, and validated with the expected output.

EXP : 06 - Cyclomatic Complexity

AIM

To calculate the **Cyclomatic Complexity** of a flow graph.

FLOWCHART



SAMPLE INPUT

Number of Edges (E) = 17

Number of Nodes (N) = 13

Number of Predicate Nodes = 5

SAMPLE OUTPUT

Cyclomatic Complexity = 6

Number of Independent Paths = 6

Minimum Test Cases Required = 6

RESULT

Thus, the Cyclomatic Complexity of the given flow graph is calculated as 6. The graph contains 6 independent paths, and a minimum of 6 test cases are required for complete path coverage.

